

本小节内容

数组的访问越界

数组的传递

1 数组的访问越界

下面借助一个数组的实例来掌握数组元素的赋值、访问越界。下例中给出了该例的全部代码。

【例】一维数组的存储及访问越界。

```
#include <stdio.h>

//数组越界

int main()

{

    int a[5]={1,2,3,4,5}; //定义数组时，数组长度必须固定

    int j=20;

    int i=10;

    a[5]=6; //越界访问

    a[6]=7; //越界访问会造成数据异常

    printf("i=%d\n",i); //i 发生改变

    return 0;

}
```

下图显示了代码运行情况。如下图所示，在第 10 行左键打上断点，然后单击“小虫子”按钮，在内存视图依次输入&j、&a、&i 来查看整型变量 j、整型数组 a、整型变量 i 的地址，即可看到三个变量的地址，这里就像我们给衣柜的每个格子的编号，第一格、第二格……一直到柜子的最后一格。操作系统对内存中的每个位置也给予一个编号，对于 Windows 32 位控制台应用程序来说，这个编号的范围是从 0x00 00 00 00 到 0xFF FF FF FF，总计为 2 的 32 次方，大小为 4G。这些编号称为地址（我们是 64 位程序，地址显示的是 64 位）。

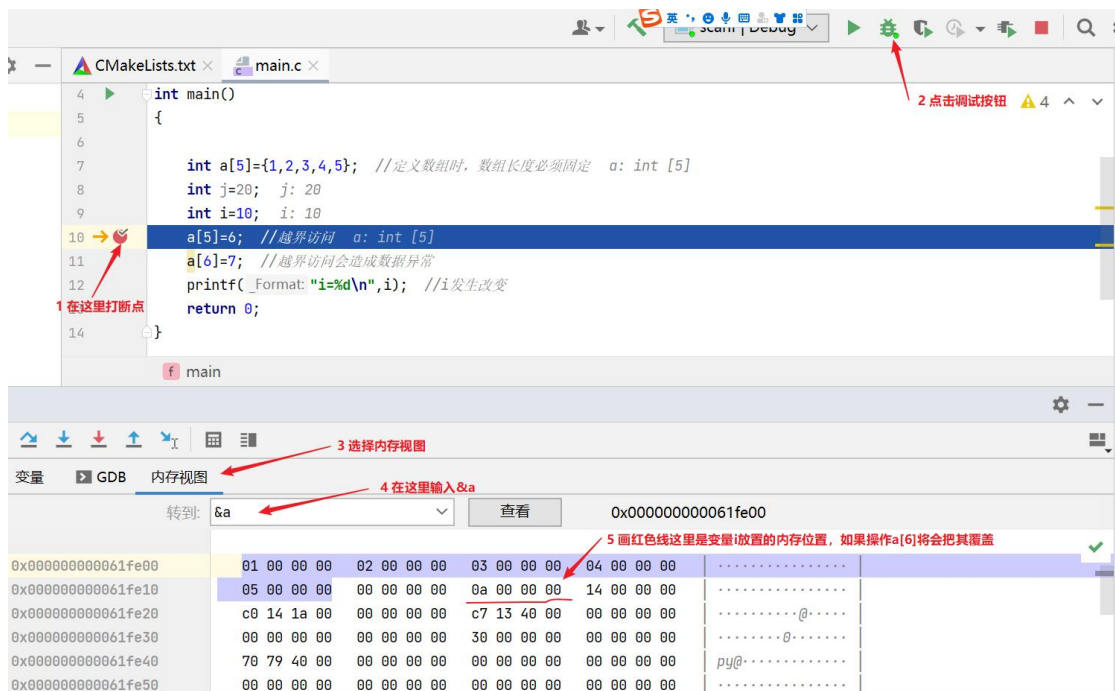


图 代码运行情况

在变量窗口中输入 `sizeof(a)`, 可以看到数组 `a` 的大小为 20 字节, 计算方法其实就是 `sizeof(int)*5`: 数组中有 5 个整型元素, 每个元素的大小为 4 字节, 所以共有 20 字节。访问元素的顺序是依次从 `a[0]` 到 `a[4]`, `a[5]=6`、`a[6]=7` 均为访问越界。下图显示了代码运行情况, 从中看出, 执行到第 12 行时, 变量 `i` 的值被修改了, 这就是访问越界的危险性——未对变量 `i` 赋值, 其值却发生了改变!

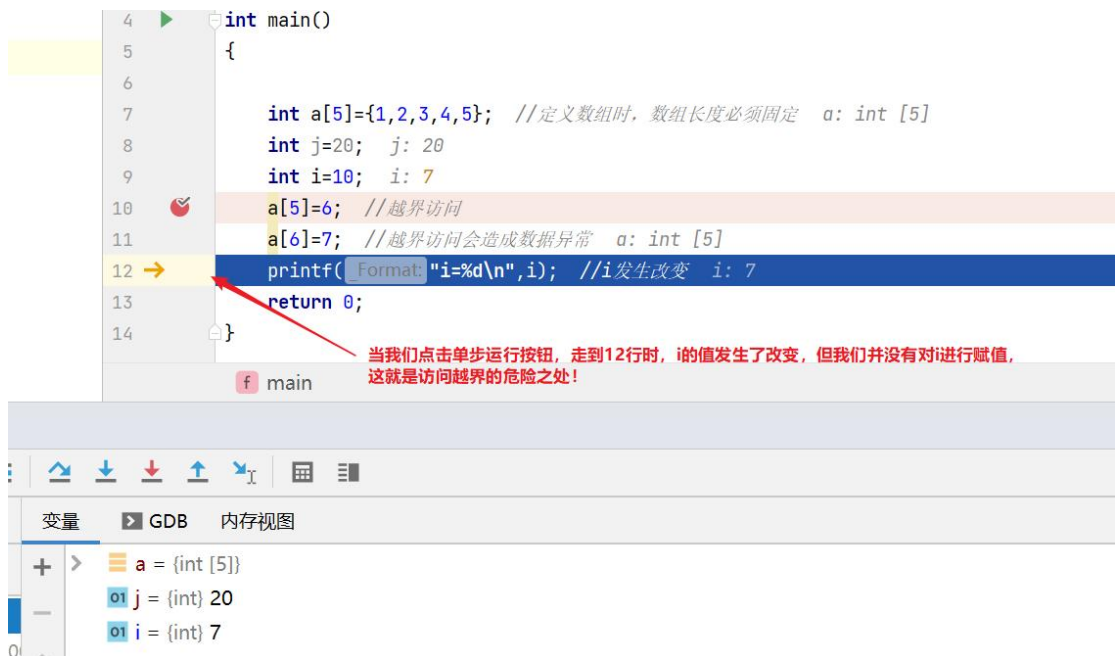


图 代码运行情况 2

数组另一个值得关注的地方是, 编译器并不检查程序对数组下标的引用是否在数组的合法范围内。这种不加检查的行为有好处也有坏处, 好处是不需要浪费时间对有些已知正确的数组下标进行检查, 坏处是这样做将无法检测出无效的下标引用。一个良好的经验法则是: 如果下标值

是通过那些已知正确的值计算得来的,那么就无须检查;如果下标值是由用户输入的数据产生的,那么在使用它们之前就必须进行检查,以确保它们位于有效范围内。

2 数组的传递

```
#include <stdio.h>

//一维数组的传递, 数组长度无法传递给子函数
//C 语言的函数调用方式是值传递
void print(int b[],int len)
{
    int i;
    for(i=0;i<len;i++)
    {
        printf("%3d",b[i]);
    }
    b[4]=20; //在子函数中修改数组元素
    printf("\n");
}

//数组越界
//一维数组的传递
#define N 5
int main()
{
    int a[5]={1,2,3,4,5}; //定义数组时, 数组长度必须固定
    print(a,5);
    printf("a[4]=%d\n",a[4]); //a[4]发生改变
    return 0;
}
```

如下图 1 所示, 在第 21 行点击向下箭头, 进入 print 函数, 这时会发现数组 b 的大小变为 8 字节, 如下图 2 所示, **这是因为一维数组在传递时, 其长度是传递不过去的, 所以我们通过 len 来传递数组中的元素个数**。实际数组名中存储的是数组的首地址, 在调用函数传递时, 是将数组的首地址给了变量 b (其实变量 b 是指针类型, 具体原理会在指针节讲解), 在 b[] 的方括号中填写任何数字都是没有意义的。这时我们在 print 函数内修改元素 b[4]=20, 可以看到数组 b 的起始地址和 main 函数中数组 a 的起始地址相同, 即二者在内存中位于同一位置, 当函数执行结束时, 数组 a 中的元素 a[4]就得到了修改。



【图 1】 代码运行情况 3



【图 2】 代码运行情况 4